

CPSC 304 Project Cover Page

Milestone #: 2

Date: Feb 22, 2026

Group Number: 45

Name	Student Number	CS Alias (Userid)	Preferred E-mail Address
Dali Li	86989274	k3w4m	ltylitarianyu@hotmail.com
Esther Zhang	75639872	c5k3q	mhua4268@gmail.com
Jiyang Zhang	75224436	q5v3a	zjiyang73@gmail.com

By typing our names and student numbers in the above table, we certify that the work in the attached assignment was performed solely by those whose names and student IDs are included above.

In addition, we indicate that we are fully aware of the rules and consequences of plagiarism, as set forth by the Department of Computer Science and the University of British Columbia

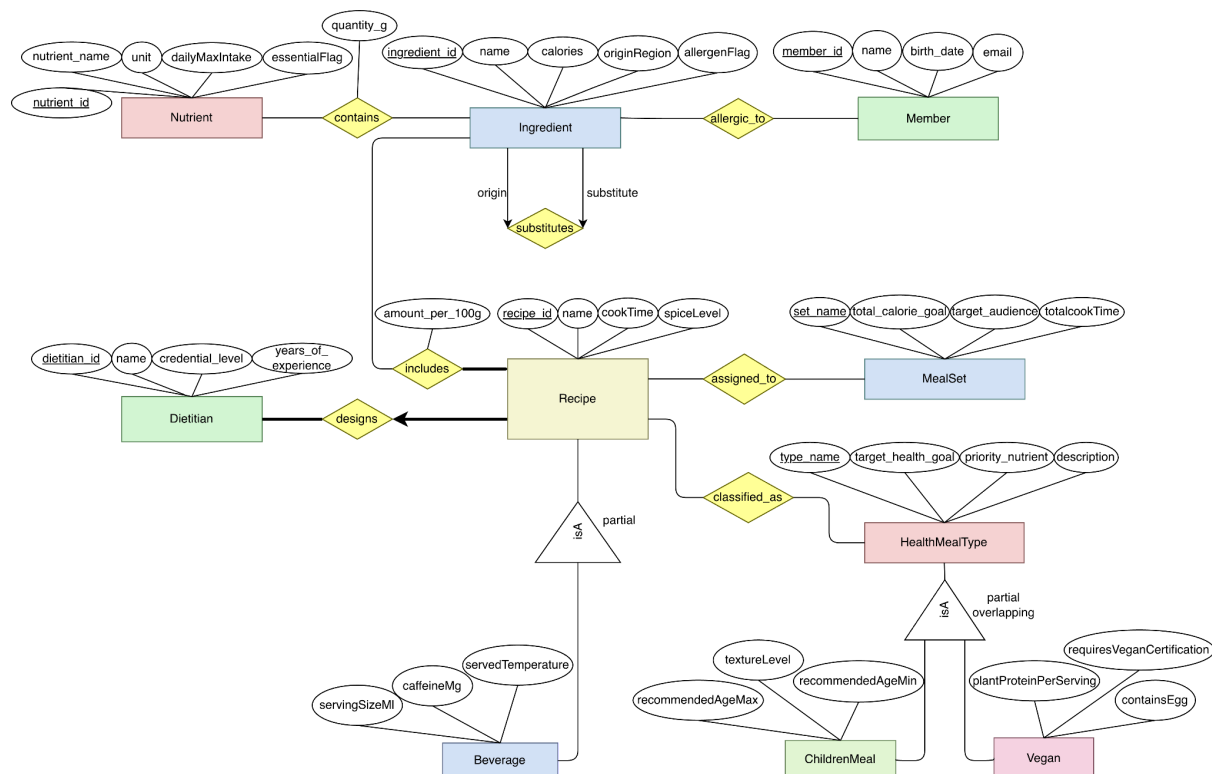
Part 1 – Project Description

This project is a healthy meal planning system that helps users create meals based on nutrition, dietary preferences, and allergies. It tracks ingredients, nutrients, and recipes designed by dietitians, and supports special health categories such as vegan and children's meals. The system enables practical and informed meal planning for individuals and families.

Part 2 – Programming Language / Database

- **Programming Language:** JavaScript
- **Database System:** Oracle

Part 3 – ER Diagram



We changed Nutrient to use nutrient_id as the primary key instead of nutrient_name. We keep nutrient_name as a non-key attribute and set it to NOT NULL, and update related tables (IngredientContainsNutrient) to reference nutrient_id as the foreign key. This makes the key stable and keeps joins consistent with other tables that use integer IDs.

Part 4 – Relational Schema

- Nutrient(nutrient_id: int, nutrient_name: varchar[100], unit: varchar[50], dailyMaxIntake: int, essentialFlag: boolean)
PK: nutrient_id
nutrient_name(not null)
- IngredientContainsNutrient(nutrient_id: int, ingredient_id: int, quantity_g: decimal)
PK: nutrient_id, ingredient_id
FK: nutrient_id → Nutrient(nutrient_id), ingredient_id → Ingredient(ingredient_id)
quantity_g(not null)
- Ingredient(ingredient_id: int, name: varchar[100], calories: int, originRegion: varchar[100], allergenFlag: boolean)
PK: ingredient_id
name(not null)
- IngredientSubstitutesIngredient(original_ingredient_id: int, substitute_ingredient_id: int)
PK: original_ingredient_id
CK: substitute_ingredient_id
FK: original_ingredient_id → Ingredient(ingredient_id), substitute_ingredient_id → Ingredient(ingredient_id)
- Member(member_id: int, name: varchar[100], birth_date: date, email: varchar[150])
PK: member_id;
CK: email
name(not null)
- RecipeIncludesIngredient (recipe_id: int, ingredient_id: int, amount_per_100g: decimal)
PK: recipe_id, ingredient_id; FK: recipe_id → Recipe(recipe_id),
ingredient_id → Ingredient(ingredient_id)
amount_per_100g(not null)
- Beverage (recipe_id: int, servingSizeMl: decimal, caffeineMg: decimal, servedTemperature: varchar[20])
PK: recipe_id; FK: recipe_id → Recipe(recipe_id)
- MealSet (set_name: varchar[20], total_calorie_goal: int, target_audience: varchar[20], price: decimal)
PK: set_name
- Recipe(recipe_id: int, name: varchar[20], cookTime: int, spiceLevel: varchar[20], price: decimal, dietitian_id: int)
PK: recipe_id; FK: dietitian_id → Dietitian(dietitian_id)
name (not null)
- Dietitian(dietitian_id: int, name: varchar[50], credential_level: varchar[30], year_of_experience: int)

PK: dietitian_id;
name (not null)

- HealthMealType(type_name: varchar[50], target_health_goal: varchar[100], priority_nutrient: varchar[50], description: varchar[255])
PK: type_name
- ChildrenMeal(type_name: varchar[50], recommendedAgeMin: int, recommendedAgeMax: int, textureLevel: varchar[50])
PK: type_name; FK: type_name→ HealthMealType(type_name)
- VeganMeal(type_name: varchar[50], plantProteinPerServing: decimal, requiresVeganCertification: boolean, containsEgg: boolean)
PK: type_name; FK: type_name→ HealthMealType(type_name)

Part 5 – Functional Dependencies (FDs)

1. Nutrient(nutrient_id: int, nutrient_name: varchar[100], unit: varchar[50], dailyMaxIntake: int, essentialFlag: boolean)
 - FD(1) nutrient_id -> nutrient_name, unit, dailyMaxIntake, essentialFlag
 2. IngredientContainsNutrient(nutrient_id: int, ingredient_id: int, quantity_g: decimal)
 - FD(2) (nutrient_id, ingredient_id) -> quantity_g
 3. Ingredient(ingredient_id: int, name: varchar[100], calories: int, originRegion: varchar[100], allergenFlag: boolean)
 - FD(3) ingredient_id -> name, calories, originRegion, allergenFlag
 4. IngredientSubstitutesIngredient(original_ingredient_id: int, substitute_ingredient_id: int)
 - FD(4) original_ingredient_id -> substitute_ingredient_id
 - FD(5) substitute_ingredient_id -> original_ingredient_id
 5. Member(member_id: int, name: varchar[100], birth_date: date, email: varchar[150])
 - FD(6) member_id -> name, birth_date, email
 - FD(7) email -> member_id, name, birth_date
 6. RecipeIncludesIngredient (recipe_id: int, ingredient_id: int, amount_per_100g: decimal)
 - FD(8) (recipe_id, ingredient_id) -> amount_per_100g
 7. Beverage (recipe_id: int, servingSizeMl: decimal, caffeineMg: decimal, servedTemperature: varchar[20])
 - FD(9) recipe_id -> servingSizeMl, caffeineMg, servedTemperature
 8. MealSet (set_name: varchar[20], total_calorie_goal: int, target_audience: varchar[20], price: decimal)
 - FD(10) set_name -> total_calorie_goal, target_audience, price
 9. Recipe(recipe_id: int, name: varchar[20], cookTime: int, spiceLevel: varchar[20], price: decimal, dietitian_id: int)
 - FD(11) recipe_id -> name, cookTime, spiceLevel, price, dietitian_id
- (NOTE: The relationship “Designs” between Dietitian and Recipe is a 1:N relationship. Therefore, the primary key of Dietitian (dietitian_id) is included as a foreign key in the Recipe relation. No separate relation is needed for the relationship.)

10. Dietitian (dietitian_id: int, name: varchar[50], credential_level: varchar[30], year_of_experience: int)
 - FD(12) dietitian_id → name, credential_level, year_of_experience
11. HealthMealType (type_name: varchar[50], target_health_goal: varchar[100], priority_nutrient: varchar[50], description: varchar[255])
 - FD(13) type_name → target_health_goal, priority_nutrient, description
 - FD(14) priority_nutrient → target_health_goal
12. ChildrenMeal (type_name: varchar[50], recommendedAgeMax: int, recommendedAgeMin: int, textureLevel: varchar[50])
 - FD(15) type_name → recommendedAgeMax, recommendedAgeMin, textureLevel
13. VeganMeal (type_name: varchar[50], plantProteinPerServing: decimal, requiresVeganCertification: boolean, containsEgg: boolean)
 - FD(16) type_name → plantProteinPerServing, requiresVeganCertification, containsEgg

Part 6 – Normalization

Original Relation (Before Decomposition)

HealthMealType(type_name: varchar[50], target_health_goal: varchar[100], priority_nutrient: varchar[50], description: varchar[255])

Functional Dependencies:

FD(13): type_name → target_health_goal, priority_nutrient, description

FD(14): priority_nutrient → target_health_goal

PK: type_name

Step 1: Check for BCNF Violations

A relation is in BCNF if for every non-trivial functional dependency $X \rightarrow b$, X is a superkey.

FD(13): type_name → target_health_goal, priority_nutrient, description

Since type_name is the primary key, this dependency satisfies BCNF.

FD(14): priority_nutrient → target_health_goal

priority_nutrient is not a superkey (it does not determine type_name or description).

Therefore, FD(14) violates BCNF.

Additionally, there exists a transitive dependency:

type_name → priority_nutrient

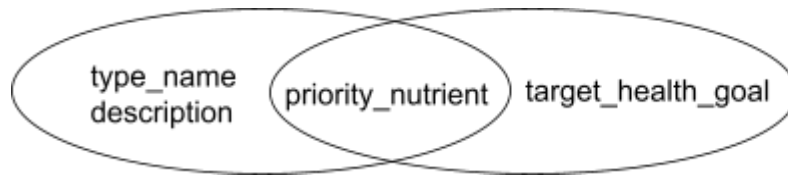
priority_nutrient → target_health_goal

Therefore: type_name → target_health_goal (transitive)

The relation is not in BCNF.

Step 2: Decomposition Based on FD(14)

Decompose using the violating dependency:



R1 = NutrientGoal(priority_nutrient: varchar[50], target_health_goal: varchar[100])

PK: priority_nutrient

FD: priority_nutrient → target_health_goal

R1 is in BCNF.

R2 = MealType(type_name: varchar[50], description: varchar[255], priority_nutrient: varchar[50])

PK: type_name

FD: type_name → description, priority_nutrient

R2 is in BCNF.

Now, all relations are in BCNF.

In addition, we checked the functional dependencies for all other relations in our schema, and they all satisfy BCNF (or 3NF where applicable), so no further decompositions were needed.

Post-Normalization Tables (BCNF)

- Nutrient(nutrient_id: int, nutrient_name: varchar[100], unit: varchar[50], dailyMaxIntake: int, essentialFlag: boolean)
PK: nutrient_id
nutrient_name(not null)
- IngredientContainsNutrient(nutrient_id: int, ingredient_id: int, quantity_g: decimal)
PK: nutrient_id, ingredient_id
FK: nutrient_id → Nutrient(nutrient_id), ingredient_id → Ingredient(ingredient_id)
quantity_g(not null)
- Ingredient(ingredient_id: int, name: varchar[100], calories: int, originRegion: varchar[100], allergenFlag: boolean)
PK: ingredient_id
name(not null)
- IngredientSubstitutesIngredient(original_ingredient_id: int, substitute_ingredient_id: int)
PK: original_ingredient_id
CK: substitute_ingredient_id
FK: original_ingredient_id → Ingredient(ingredient_id),

substitute_ingredient_id → Ingredient(ingredient_id)

- Member(member_id: int, name: varchar[100], birth_date: date, email: varchar[150])
PK: member_id;
CK: email
name(not null)
- RecipeIncludesIngredient (recipe_id: int, ingredient_id: int, amount_per_100g: decimal)
PK: recipe_id, ingredient_id; FK: recipe_id→Recipe(recipe_id),
ingredient_id→Ingredient(ingredient_id)
amount_per_100g(not null)
- Beverage (recipe_id: int, servingSizeMl: decimal, caffeineMg: decimal, servedTemperature: varchar[20])
PK: recipe_id; FK: recipe_id→Recipe(recipe_id)
- MealSet (set_name: varchar[20], total_calorie_goal: int, target_audience: varchar[20], price: decimal)
PK: set_name
- Recipe(recipe_id: int, name: varchar[20], cookTime: int, spiceLevel: varchar[20], price: decimal, dietitian_id: int)
PK: recipe_id; FK: dietitian_id→Dietitian(dietitian_id)
name (not null)
- Dietitian(dietitian_id: int, name: varchar[50], credential_level: varchar[30], year_of_experience: int)
PK: dietitian_id;
name(not null)
- NutrientGoal (priority_nutrient: varchar[50], target_health_goal: varchar[100])
PK: priority_nutrient;
- MealType (type_name: varchar[50], description: varchar[255], priority_nutrient: varchar[50])
PK: type_name; FK: priority_nutrient → NutrientGoal (priority_nutrient);
- ChildrenMeal(type_name: varchar[50], recommendedAgeMin: int, recommendedAgeMax: int, textureLevel: varchar[50])
PK: type_name; FK: type_name→ MealType(type_name)
- VeganMeal(type_name: varchar[50], plantProteinPerServing: decimal, requiresVeganCertification: boolean, containsEgg: boolean)
PK: type_name; FK: type_name→ MealType(type_name)

Part 7 – SQL DDL

-- Table: Nutrient

-- Stores nutrient information

```
CREATE TABLE Nutrient (  
    nutrient_id INT,  
    nutrient_name VARCHAR(100) NOT NULL,  
    unit VARCHAR(50),  
    dailyMaxIntake INT,  
    essentialFlag NUMBER(1),  
    PRIMARY KEY (nutrient_id)  
);
```

-- Table: Ingredient

-- Stores ingredient information

```
CREATE TABLE Ingredient (  
    ingredient_id INT,  
    name VARCHAR(100) NOT NULL,  
    calories INT,  
    originRegion VARCHAR(100),  
    allergenFlag NUMBER(1),  
    PRIMARY KEY (ingredient_id)  
);
```

-- Table: IngredientContainsNutrient

-- Junction table between Ingredient and Nutrient

```
CREATE TABLE IngredientContainsNutrient (  
    nutrient_id INT,  
    ingredient_id INT,  
    quantity_g DECIMAL NOT NULL,  
    PRIMARY KEY (nutrient_id, ingredient_id),  
    FOREIGN KEY (nutrient_id) REFERENCES Nutrient(nutrient_id)  
        ON DELETE CASCADE,  
    FOREIGN KEY (ingredient_id) REFERENCES Ingredient(ingredient_id)  
        ON DELETE CASCADE  
);
```

-- ON DELETE CASCADE: if a nutrient is deleted, all ingredientcontainsnutrient rows referencing it are removed.

-- ON DELETE CASCADE: if an ingredient is deleted, all ingredientcontainsnutrient rows referencing it are removed.

-- We do not set ON UPDATE because Oracle does not support it; primary keys are immutable, so updates are not needed

-- Table: IngredientSubstitutesIngredient

-- Stores 1:1 substitution relationships between ingredients

```
CREATE TABLE IngredientSubstitutesIngredient (  
    original_ingredient_id INT,
```



```

        substitute_ingredient_id INT,
        PRIMARY KEY (original_ingredient_id),
        UNIQUE (substitute_ingredient_id),
        FOREIGN KEY (original_ingredient_id) REFERENCES Ingredient(ingredient_id)
            ON DELETE CASCADE,
        FOREIGN KEY (substitute_ingredient_id) REFERENCES Ingredient(ingredient_id)
            ON DELETE CASCADE
    );
    -- ON DELETE CASCADE: if either ingredient is deleted, the substitution row is
    removed.
    -- We do not set ON UPDATE because Oracle does not support it; primary keys are
    immutable, so updates are not needed

-- Table: Member
-- Stores member information
CREATE TABLE Member (
    member_id INT,
    name VARCHAR(100) NOT NULL,
    birth_date DATE,
    email VARCHAR(150) NOT NULL,
    PRIMARY KEY (member_id),
    UNIQUE (email)
);

-- Table: Dietitian
-- Stores dietitian information
CREATE TABLE Dietitian (
    dietitian_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    credential_level VARCHAR(30),
    year_of_experience INT
);

-- Table: Recipe
-- Stores recipe information, including Dietitian FK
CREATE TABLE Recipe(
    recipe_id    INTEGER,
    name         VARCHAR(20) NOT NULL,
    cookTime     INTEGER,
    spiceLevel   VARCHAR(20),
    price        DECIMAL,
    dietitian_id INTEGER,
    PRIMARY KEY (recipe_id),
    FOREIGN KEY (dietitian_id) REFERENCES Dietitian(dietitian_id)
);
    -- We do not set ON DELETE CASCADE here because if a Dietitian is removed from
    the system, we want to retain their Recipes

```

```
-- Table: RecipeIncludesIngredient
-- Junction table for Recipe and Ingredient with amount per 100g
CREATE TABLE RecipeIncludesIngredient(
    recipe_id    INTEGER,
    ingredient_id INTEGER,
    amount_per_100g DECIMAL NOT NULL,
    PRIMARY KEY (recipe_id, ingredient_id),
    FOREIGN KEY (recipe_id) REFERENCES Recipe(recipe_id)
        ON DELETE CASCADE,
    FOREIGN KEY (ingredient_id) REFERENCES Ingredient(ingredient_id)
);
-- ON DELETE CASCADE: if a recipe is deleted, the corresponding rows in this table
are removed.
-- We do not set ON UPDATE because Oracle does not support it; primary keys are
immutable, so updates are not needed
```

```
-- Table: Beverage
-- Subclass of Recipe storing beverage-specific attributes
CREATE TABLE Beverage(
    recipe_id    INTEGER,
    servingSizeMl DECIMAL,
    caffeineMg    DECIMAL,
    servedTemperature VARCHAR(20),
    PRIMARY KEY (recipe_id),
    FOREIGN KEY (recipe_id) REFERENCES Recipe(recipe_id)
        ON DELETE CASCADE
);
-- ON DELETE CASCADE: if a recipe is deleted, the corresponding beverage row is
removed.
-- We do not set ON UPDATE because Oracle does not support it; primary keys are
immutable, so updates are not needed
```

```
-- Table: MealSet
-- Stores meal set information
CREATE TABLE MealSet(
    set_name      VARCHAR(20),
    total_calorie_goal INTEGER,
    target_audience VARCHAR(20),
    price         DECIMAL,
    PRIMARY KEY (set_name)
);
```

```
-- Table: NutrientGoal
-- Stores target health goal for a priority nutrient
CREATE TABLE NutrientGoal (
    priority_nutrient VARCHAR(50) PRIMARY KEY,
    target_health_goal VARCHAR(100)
);
```

```
-- Table: MealType
-- Stores health meal type information
CREATE TABLE MealType (
    type_name VARCHAR(50) PRIMARY KEY,
    description VARCHAR(255),
    priority_nutrient VARCHAR(50),
    FOREIGN KEY (priority_nutrient)
        REFERENCES NutrientGoal(priority_nutrient)
);
-- No ON DELETE CASCADE: deleting a NutrientGoal should not automatically
remove related MealType records to prevent unintended data loss.
-- We do not set ON UPDATE because Oracle does not support it; primary keys are
immutable, so updates are not needed
```

```
-- Table: ChildrenMeal
-- Stores children-specific meal information
CREATE TABLE ChildrenMeal (
    type_name VARCHAR(50) PRIMARY KEY,
    recommendedAgeMin INT,
    recommendedAgeMax INT,
    textureLevel VARCHAR(50),
    FOREIGN KEY (type_name)
        REFERENCES MealType(type_name)
        ON DELETE CASCADE
);
-- ON DELETE CASCADE: if a meal type is deleted from MealType, the corresponding
row in this table is removed.
-- We do not set ON UPDATE because Oracle does not support it; primary keys are
immutable, so updates are not needed
```

```
-- Table: VeganMeal
-- Stores vegan meal-specific information
CREATE TABLE VeganMeal (
    type_name VARCHAR(50) PRIMARY KEY,
    plantProteinPerServing DECIMAL,
    requiresVeganCertification NUMBER(1),
    containsEgg NUMBER(1),
    FOREIGN KEY (type_name)
        REFERENCES MealType(type_name)
        ON DELETE CASCADE
);
-- ON DELETE CASCADE: if a meal type is deleted from MealType, the corresponding
row in this table is removed.
-- We do not set ON UPDATE because Oracle does not support it; primary keys are
immutable, so updates are not needed.
```

Part 8 – Acknowledgment About Use Of AI

We did not use AI for Milestone 2.